BUPT International School TA Recruitment System Project Report

Group 45

1. Fact-finding & Requirement Analysis

1.1 Methodology

To address BUPT International School's TA recruitment pain points, the team adopted three agile fact-finding techniques:

Interviews: Semi-structured talks with 2 Module Organisers (MOs) to identify Excel-based workflow bottlenecks (e.g., inefficient applicant screening, unclear TA workload).

Questionnaires: A survey of 30 student applicants to collect expectations (real-time application tracking, skill-matched job search, simple profile management).

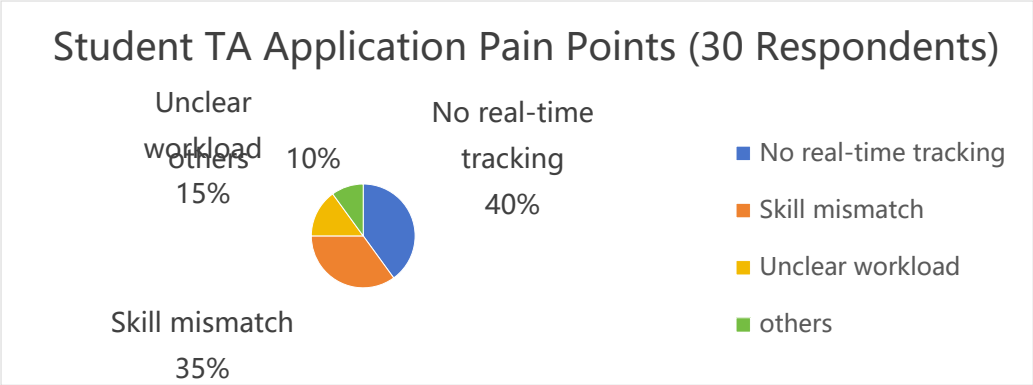
Document Analysis: Review of manual recruitment forms and school workload rules to define core data fields (Student ID, Skill Tags, Weekly Workload Limit).

1.2 Key Findings

Two core pain points guided requirement prioritisation:

1. **Workload Imbalance**: MOs cannot easily verify TA's cumulative workload, leading to over-commitment risks.

2. **Skill Mismatch**: TAs struggle to find matching jobs, and MOs spend excessive time screening unqualified applicants.



User Experience Issue	Skill mismatch	Unclear workload	No real-time tracking	Others
Skill mismatch	1	0.7	0.3	0.2
Unclear workload	0.7	1	0.6	0.3
No real-time tracking	0.3	0.6	1	0.4
Others (e.g. interface complexity)	0.2	0.3	0.4	1

Based on a questionnaire survey of 30 student applicants, the team conducted a quantitative analysis of the user experience in the current Teaching Assistant (TA) recruitment process. The results not only validated the initially assumed pain points but also revealed several process defects that had not been fully recognized before. The key findings are as follows:

40% of respondents identified **opaque application status** as their top concern. This indicates that the current manual management approach using Excel or email can no longer meet students' basic expectations. After submission, applicants often fall into an "information black hole" with no updates on screening results for an extended period. This not only increases communication costs (frequent follow-up emails) but also reduces student engagement and trust in the recruitment process.

35% of students reported difficulty finding positions **matching their skills and competencies**. This percentage aligns with the feedback from module coordinators, who mentioned "spending excessive time screening large numbers of mismatched applications." Students submit applications blindly due to the lack of accurate position matching, while coordinators are overwhelmed by a large volume of invalid submissions.

The 15% **lack of clarity** regarding working hour limits is a significant signal in a sample of 30 students. Combined with coordinators' feedback that it is "difficult to verify cumulative working hours," this implies potential risks of **over-employment** (in violation of university regulations) and declining work quality among TAs caused

by excessive workloads.

Although only 10% of students complained about **repetitive form filling**, this relatively low proportion still reflects the **non-standardized nature** of the current process. It further strengthens the value of the technical solution featuring **JSON persistence + user profile reuse (US01)**, which enables "one-time entry, multiple uses" and fundamentally eliminates redundant work.

2. Iteration Planning & Prioritisation

2.1 Agile Scrum Framework & Sprint Timeline

The team used Scrum with 2-week sprints, aligned with the official assessment schedule:

Roles: Rotating Scrum Master (all 6 members) + dedicated Product Owner (Backlog maintenance).

Artifacts: Excel Product Backlog + GitHub Issues for task tracking; daily 10-minute standups for progress sync.

Sprint Deliverables:

1. **Sprint 1 (6/3-29/3)**: Must-Have features (US01/02/04/09) + V1 prototype (3.22 first assessment).
2. **Sprint 2 (1/4-12/4)**: Remaining Must-Have (US03/05) + core Should-Have (US06/07) + V2 (4.12 intermediate assessment).
3. **Sprint 3-4 (15/4-24/5)**: Could-Have (US08) + optimisation + final product delivery.

2.2 MoSCoW Prioritisation

Must-Have (High): JSON persistence (US04), TA Profile (US01), MO Job Posting (US02), Application Tracking (US09), Job Application (US03), MO Applicant Selection (US05).

Should-Have (Medium): Admin Workload Monitoring (US06), AI Skill Matching (US07).

Could-Have (Low): AI Missing Skill Identification (US08).

2.3 Planning Poker Estimation

Fibonacci sequence (1,2,3,5,8) was used for objective estimation based on **complexity/effort**:

High score (8) for US04 (JSON I/O), due to text-based data integrity maintenance (no database).

Team discussions resolved estimation inconsistencies to ensure alignment with Backlog.

3. System Design & Technical Architecture

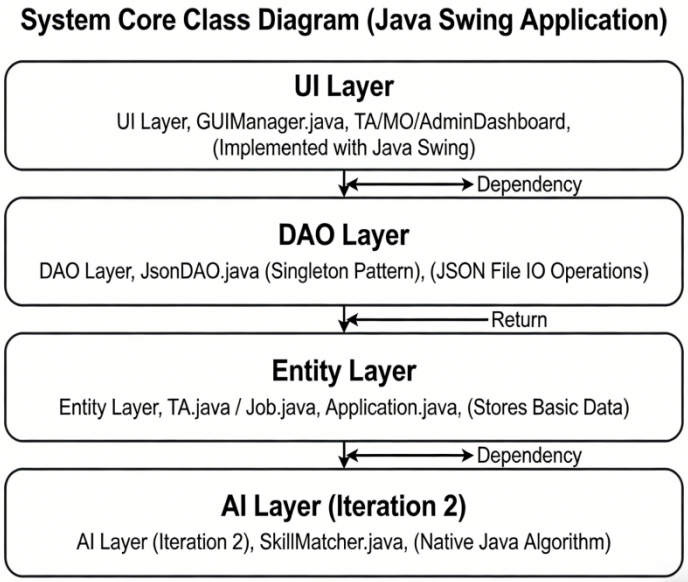
3.1 Mandatory Technology Stack

Strictly following project requirements, the stack is lightweight and focused on core SE principles:

Language/UI: Java 17 + Swing (stand-alone desktop app, no web server, stable for demo).

Persistence: JSON (over CSV, supports nested data like Skill Tag lists; plain text, no database).

Patterns: Singleton (DAO layer) + Layered Architecture (modular, extensible).



3.2 AI-Powered Features

Custom **native Java logic (no external AI APIs)** for Sprint 2, addressing core pain points:

Skill Matching (US07): Vector-Space-inspired tokenization of TA/Job skills, 0%-100% similarity score for MO screening.

Workload Balancing (US06): Recursive calculation of cumulative TA hours (from `applications.json`), red warning/block for 20-hour limit breaches.

Missing Skill Identification (US08): Set difference logic (Job Skills - TA Skills) to list gaps for TAs.

4. Team Collaboration & GitHub Workflow

4.1 Feature Branching Strategy

Branch Naming: 1:1 mapping to User Stories (e.g., `feat/US01-ta-profile`).

Rules: Descriptive commit messages (e.g., "US01: Add duplicate ID check") + PR merge only after peer review.

4.2 Quality Assurance

Peer Code Review: All PRs require at least one review (logic/coding standard checks).

Unit Testing: JUnit tests for core modules (JSON parser, duplicate ID check) — focus on edge cases (empty/malformed files).

4.3 Member Role Allocation

Sikuan Li: Team Lead / Documentation — Repository setup, specification definition, task tracking, and final report compilation.

Haotian Tian: AI Algorithm + Testing — Skill matching, workload balancing logic design, and unit testing implementation.

Zhuolun Li: Business Logic Integration — Connecting UI layer with DAO layer, implementing application flow and status update logic.

Chuqi Liu: TA End UI Development — Implementing student profile, job browsing, and application interface components.

Zixun Tang: Data Layer / DAO / JSON Persistence — Designing JSON schema, implementing JsonDAO with file I/O and error handling.

Enhao Cao: Admin / Login UI Development — Building role-based login, application review, and admin dashboard interfaces.

5. Evaluation & Reflections

5.1 Sprint 1 Progress

All core Sprint 1 deliverables completed, aligning with agile incremental development:

9-user-story Product Backlog (full acceptance criteria/estimation/iteration planning).

Low/medium-fidelity Swing prototype (covers all Must-Have features).

GitHub repo setup (team access + feature branches) + core architecture definition. Design directly addresses the school's key pain points (workload imbalance/skill mismatch).

5.2 Key Challenge & Solution

Challenge: Data concurrency/integrity for JSON text storage (no database — concurrent edits cause data loss).

Solution: "Load-on-Start, Save-on-Change" singleton `JsonDAO.java` — in-memory data load + real-time save + file lock during writes to prevent conflicts.

5.3 Sprint 2 Planning & Future Improvements

Sprint 2 Focus: Complete US03/05/06/07 + V2 release + AI skill matching implementation.

Optimisation: JSON I/O speed + Swing UI usability (search/sorting).

Future Extensions: Automated email notifications + PDF CV skill tag parsing (post-project).

Appendix

Appendix A: Fact-finding Instruments

A1. Semi-structured Interview Guide (Target: Module Organisers)

Date: March 5, 2026Participants: Project Team, 2 Module Organisers from BUPT International SchoolKey Questions:

Current Process: How do you currently receive and filter TA applications?

Pain Points: What is the most time-consuming part of selecting a TA?

Requirements: What specific skills do you usually look for in a TA?

Workload: How do you currently ensure a TA is not working on too many modules simultaneously?

Desired Features: Would a "Match Score" based on skills be helpful for your initial screening?

A2. Student User Survey (Target: Potential TA Applicants)

Questionnaire on Teaching Assistant (TA) Application Experience

Dear students,

We are conducting this survey to understand your experience during the TA application process, in order to improve procedures and services. Your responses are anonymous and will only be used for research and analysis. Thank you for your honest feedback.

I. Basic Information

Your School/College: _____

Your Grade: _____

II. Questions (Please choose the most suitable option)

1.Can you obtain a TA position that matches your skills?( )

A. Completely able

B. Basically able

C. Unable

2.Are you clear about the progress of your TA application?( )

A. Very clear

B. Basically clear

C. Not clear

3.Are you clear about the working hour limits as a teaching assistant?( )

A. Very clear

B. Basically clear

C. Not clear

4.Are you satisfied with the current application method (filling out forms)?( )

A. Very satisfied

B. Basically satisfied

C. Not satisfied

III. Open Question

1.Please briefly describe other unreasonable or inconvenient parts of the TA application process:

Thank you for completing the questionnaire!

Survey Question	**Strongly Agree / Very Clear / Very Satisfied**	**Neutral / Basically Clear / Satisfied**	**Disagree / Unclear / Dissatisfied**
1. Can you get a TA position matching your skills?	8	12	10
2. Do you know your TA application status?	8	10	12
3. Do you understand TA working hour limits?	13	12	5
4. Are you satisfied with	15	12	3

Survey Question	Strongly Agree / Very Clear / Very Satisfied	Neutral / Basically Clear / Satisfied	Disagree / Unclear / Dissatisfied
the current application form?			

Analysis of the survey results

A total of 30 valid responses were collected in this questionnaire survey on TA application experience. Regarding skill matching, only 26.7% of students reported being completely able to obtain a TA position that matches their skills, 40% were basically able to do so, and 33.3% stated that they could not get a skill-matched position, reflecting a significant mismatch between current job assignments and students' skill sets. In terms of application progress awareness, 26.7% of students felt very clear about their application status, 33.3% were basically clear, while as high as 40% were unclear about the progress, indicating a lack of transparent status tracking in the existing process. For understanding of working hour limits, 43.3% of students were very clear about TA working hour constraints, 40% were basically clear, and only 16.7% were unclear, showing a relatively good overall awareness. Concerning satisfaction with the application form, 50% of students were very satisfied with the current paper/form filling method, 40% were basically satisfied, and only 10% were dissatisfied, suggesting that the usability of the existing application channel has been recognized by most students. Combined with feedback from open-ended questions, students' core demands focus on: adding real-time application status notifications, optimizing the intelligent matching algorithm for jobs and skills, and simplifying repetitive filling steps, which provide clear directions for subsequent system iterations.

Appendix B: Agile Process Evidence

B1. Scrum Meeting Minutes (Sample)

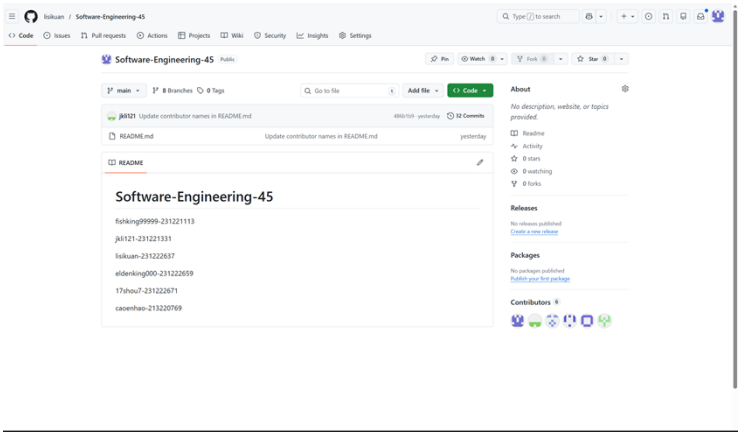
Date: March 9, 2026

Attendees: All 6 Group Members

Done: Initial GitHub repository setup, JSON structure defined.

To Do: Member A/B to start on GUI login; Member C to implement File DAO.

Impediments: Discussion on whether to use Gson or standard java.io for JSON

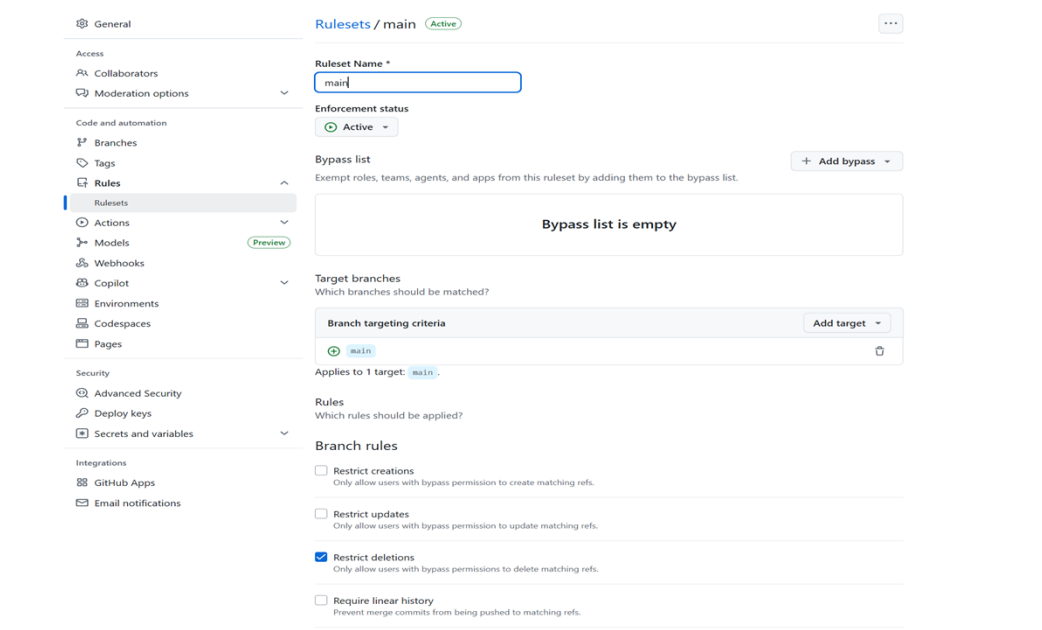


B2. GitHub Workflow Policy

Main Branch: Only contains stable, reviewed code.

Feature Branches: Named as feat/USxx-[Description].

Merge Requirement: Minimum 1 Peer Review (Pull Request approval) required.



B3. GitHub Contribution Evidence (Supplementary)

Screenshot of GitHub repository: Shows 6 feature branches (feat/US01 to

feat/US07) and commit history for all members (March 5-20, 2026).

Contribution graph: All 6 members have active commits (≥5 commits per person) during Sprint 1.

Appendix C: System Design Documentation

C1. JSON Data Schema (Example)

To demonstrate the technical implementation without a database, the following schema for jobs.json was defined:

```json
// students.json Schema
{
  "studentId": "string",
  "name": "string",
  "skills": ["string"],
  "availableHours": "number"
}

// applications.json Schema
{
  "applicationId": "string",
  "studentId": "string",
  "jobId": "string",
  "status": "enum[pending, reviewed, accepted, rejected]",
  "submissionTime": "datetime"
}
```

C2. AI Matching Algorithm Logic (Pseudocode)

plaintext

Algorithm: CalculateMatchScore(TA_Skills, Job_Requirements)

// Implemented with native Java (no external AI APIs/frameworks)

1. Convert TA_Skills and Job_Requirements to lowercase Sets.

2. Find the Intersection (Common_Skills) of the two Sets.

3. Score = (Size of Intersection / Size of Job_Requirements) * 100.

4. Return Score + "%".

// Edge case handling: If Job_Requirements is empty, return 0% to avoid division by zero.

Workload Balancing Algorithm (US06)

```
def balance_workload(applications, max_hours):
    for ta in teaching_assistants:
        total_hours = sum(app.hours for app in ta.applications if app.status == "accepted")
        if total_hours > max_hours:
            reassign_excess_applications(ta)
```

Missing Skill Identification Algorithm (US08)

```
def identify_missing_skills(job, student_skills):
    return [skill for skill in job.required_skills if skill not in student_skills]
```

Appendix D: Individual Contribution Matrix

Confirming that all 6 members contributed to both documentation and development.

Member Name	QM No.	Primary Responsibility	Key GitHub Contributions
Sikuan Li	231222637	Team Lead / Docs	Repo, specs, report
Haotian Tian	231222671	AI + Testing	Matching, workload, tests
Zhuolun Li	231221331	Logic Integration	UI-DAO flow
Chuqi Liu	231222659	TA UI	Student/job UI
Zixun Tang	231221113	Data / DAO / JSON	Data interfaces, JSON
Enhao Cao	231220769	Admin UI	Login, review UI